

# Teach Module 36: Secure Frontend Communication

## Module 36 Topics

Module 36 covers secure communication between a frontend application and backend APIs.

Core topics:

- Token-based authentication, especially JWT
- Secure API calls
- Session protection
- Frontend input validation
- XSS and CSRF awareness
- Frontend security lab

This guide uses the course document only as a module roadmap. The explanations and examples below are independent teaching material.

---

## 1. What Secure Frontend Communication Means

Frontend security is about protecting the communication between the browser and the backend.

The frontend cannot be fully trusted because users can inspect, modify, and replay browser requests. A user can open developer tools, change JavaScript values, alter HTTP requests, or call backend APIs directly.

So the goal is not to make frontend code secret. The goal is to make the frontend behave safely while the backend remains the real security authority.

A secure frontend does five big things well:

1. Authenticates users safely.
2. Sends API requests securely.
3. Protects sessions and tokens.
4. Validates user input early.
5. Avoids common browser attacks like XSS and CSRF.

Important mental model:

```
Frontend:
helps the user do the right thing

Backend:
prevents users and attackers from doing the wrong thing
```

The frontend should be careful, but the backend must be strict.

---

## 2. Token-Based Authentication

In many modern applications, after a user logs in, the backend gives the frontend a token.

A common token format is JWT, which stands for JSON Web Token.

A JWT usually has three parts:

```
header.payload.signature
```

The payload may contain claims such as:

```
{
  "sub": "user123",
  "role": "ADMIN",
  "exp": 1720000000
}
```

Important idea:

A JWT is often readable by the browser, but it must not be forgeable. The backend signs the token. The frontend may store it and send it back, but the backend verifies the token signature and expiration.

Typical login flow:

```
User logs in
Frontend sends username/password to backend
Backend verifies credentials
Backend returns token
Frontend stores token carefully
Frontend sends token with future API requests
Backend validates token on every protected request
```

Common request format:

```
Authorization: Bearer <jwt-token>
```

Example frontend request:

```
fetch("/api/orders", {
  headers: {
    Authorization: `Bearer ${accessToken}`
  }
});
```

Key rule:

The frontend should never decide whether a user is truly authorized. It can hide or show UI, but the backend must enforce permissions.

---

### 3. Secure API Calls

Every API call that carries login tokens, user data, payment data, personal information, or business data should use HTTPS.

Why HTTPS matters:

Without HTTPS:

browser -> attacker may inspect or change traffic -> server

With HTTPS:

browser -> encrypted channel -> server

Frontend API security practices:

- Always use HTTPS in production.
- Do not put secrets in frontend code.
- Do not expose private API keys in JavaScript bundles.
- Send authentication tokens only to trusted backend domains.
- Handle API errors without leaking sensitive details.
- Avoid logging tokens or personal data in the browser console.

Bad idea:

```
const API_SECRET = "super-private-key";
```

Anything bundled into frontend JavaScript can be seen by users.

Better architecture:

Frontend -> your backend -> third-party service

The backend stores secrets. The frontend talks only to your backend.

---

## 4. Session Protection

A session means the application remembers that a user is logged in.

Two common approaches:

Cookie-based session  
Token-based session

With cookies, the browser automatically sends the cookie with requests.

With bearer tokens, frontend code usually attaches the token manually.

For cookies, strong settings matter:

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Lax
```

Meaning:

- `HttpOnly` : JavaScript cannot read the cookie.
- `Secure` : cookie is sent only over HTTPS.
- `SameSite` : helps reduce CSRF risk.

For JWT or token storage, the big question is where to store the token.

Common options:

```
localStorage
sessionStorage
memory
HttpOnly cookie
```

Tradeoffs:

- `localStorage` is easy, but vulnerable if XSS happens.
- `sessionStorage` clears when the browser session closes, but is still accessible to JavaScript.
- In-memory storage is safer against persistent theft, but disappears on refresh.
- `HttpOnly` cookies are safer from JavaScript token theft, but need CSRF protection.

A mature app often uses:

```
Short-lived access token + refresh token + backend validation
```

---

## 5. Frontend Input Validation

Frontend validation improves user experience, but it is not enough for security.

Example:

```
if (!email.includes("@")) {
  showError("Enter a valid email address");
}
```

That helps the user, but attackers can bypass frontend validation by sending requests directly to the backend.

Remember:

```
Frontend validation = helpful
Backend validation = mandatory
```

Frontend should validate:

- Required fields
- Email format
- Password length rules
- Number ranges
- File type hints
- Form completeness

Backend must validate:

- Permissions
- Data ownership
- Business rules
- Dangerous characters or content
- File contents

- Database constraints
- 

## 6. XSS Awareness

XSS means Cross-Site Scripting.

It happens when attacker-controlled code runs in another user's browser.

Dangerous pattern:

```
element.innerHTML = userInput;
```

If `userInput` contains:

```
<img src=x onerror="alert('XSS')">
```

the browser may execute malicious code.

Safer pattern:

```
element.textContent = userInput;
```

Modern frameworks like React escape text by default:

```
<p>{userComment}</p>
```

But danger returns when using raw HTML features:

```
<div dangerouslySetInnerHTML={{ __html: userContent }} />
```

To reduce XSS risk:

- Prefer text rendering over raw HTML.
  - Sanitize HTML if you must render it.
  - Avoid storing tokens where JavaScript can easily read them.
  - Use Content Security Policy where appropriate.
  - Do not trust user-generated content.
- 

## 7. CSRF Awareness

CSRF means Cross-Site Request Forgery.

It happens when a malicious site tricks a logged-in browser into sending a request to your app.

Example:

```
User is logged into bank.com  
User visits evil-site.com
```

```
evil-site.com causes browser to send request to bank.com  
Browser automatically includes bank.com cookies
```

CSRF mostly affects cookie-based authentication because cookies are sent automatically.

Defenses:

- Use `SameSite` cookies.
- Use CSRF tokens for state-changing requests.
- Require custom headers for API calls.
- Validate request origin where appropriate.
- Avoid using GET requests for actions that change data.

Bad design:

```
GET /transfer-money?to=attacker&amount=1000
```

Better design:

```
POST /transfer-money  
Authorization or CSRF protection required
```

---

## 8. Practice Exercises

### Exercise 1: Secure Login Flow

Build a small frontend login page that sends credentials to a backend endpoint.

Practice goals:

- Submit username/password over `POST`.
- Store a returned access token.
- Redirect the user to a protected dashboard.
- Show friendly errors for invalid login.
- Avoid putting passwords or tokens in console logs.

Example flow:

```
/login -> submit credentials -> receive token -> /dashboard
```

### Exercise 2: Authenticated API Calls

Create a page that fetches protected data from an API.

Practice goals:

- Attach a JWT using the `Authorization` header.
- Handle `401 Unauthorized`.
- Handle `403 Forbidden`.
- Show loading, success, and error states.

Example:

```
fetch("/api/profile", {
  headers: {
    Authorization: `Bearer ${token}`
  }
});
```

Add logic like:

```
If token is missing -> send user to login
If token is expired -> show session expired message
If request succeeds -> show profile data
```

### Exercise 3: Token Expiration Handling

Simulate an expired token.

Practice goals:

- Decode the JWT payload on the frontend.
- Check the `exp` claim.
- Warn the user before expiration.
- Log the user out after expiration.

You do not need real authentication for this exercise. You can use a fake JWT-like object or mock API response.

### Exercise 4: Secure Logout

Implement logout properly.

Practice goals:

- Clear token from client storage.
- Clear user state from memory.
- Redirect to login.
- Prevent back-button access to protected pages.
- Optional: call `/api/logout` if the backend supports session invalidation.

Test:

```
Login -> visit dashboard -> logout -> try dashboard again
```

Expected result:

```
Dashboard should not show protected data after logout.
```

### Exercise 5: Frontend Input Validation

Create a registration form.

Fields:

- Full name
- Email

- Password
- Confirm password
- Phone number

Validation rules:

- Email must be valid.
- Password must be at least 8 characters.
- Password and confirmation must match.
- Phone number should allow only expected characters.
- Submit button should be disabled until valid.

Important lesson:

Frontend validation improves user experience, but backend validation is still required.

## Exercise 6: XSS Demonstration

Build a simple comment preview box.

Unsafe version:

```
preview.innerHTML = userInput;
```

Then try input like:

```
<img src=x onerror="alert('XSS')">
```

Safer version:

```
preview.textContent = userInput;
```

Practice goals:

- Understand why `innerHTML` is risky.
- Learn when escaping or sanitizing is needed.
- See how framework defaults help prevent XSS.

## Exercise 7: React XSS Safety

If using React, create a comment component:

```
function Comment({ text }) {  
  return <p>{text}</p>;  
}
```

Test with HTML-looking input:

```
<script>alert('bad')</script>
```

Then compare it with:



```
<div dangerouslySetInnerHTML={{ __html: text }} />
```

Practice goal:

Understand why React's default rendering is safer than raw HTML injection.

### Exercise 8: CSRF Awareness Simulation

Create two pages:

```
trusted-app.html  
malicious-site.html
```

Have the trusted app simulate a cookie-authenticated action like:

```
POST /api/change-email
```

Then make the malicious page try to trigger the same request.

Practice goals:

- Understand why automatically sent cookies can be risky.
- Add a fake CSRF token requirement.
- Reject requests missing the CSRF token.
- Discuss how `SameSite` cookies help.

### Exercise 9: Secure API Error Handling

Create API error responses and display them safely.

Bad frontend behavior:

```
Database connection failed: password=secret123
```

Better frontend behavior:

```
Something went wrong. Please try again.
```

Practice goals:

- Avoid exposing stack traces.
- Avoid exposing backend internals.
- Show helpful but safe messages.
- Log detailed errors only on the backend.

### Exercise 10: Mini Secure Frontend Project

Build a small Secure Notes app.

Features:

- Login page
- Protected notes page

- Add note form
- Token-based API calls
- Logout
- Input validation
- XSS-safe note rendering
- Expired session handling

Security checklist:

- No secrets in frontend code.
  - Token attached only to trusted API requests.
  - Protected pages require authentication.
  - User input is validated.
  - User-created content is rendered safely.
  - Logout clears session state.
- 

## 9. Lab: Secure Notes

### Goal

Create a frontend that lets a user log in, access protected notes, add a note, and log out while applying frontend security practices.

### What You Will Practice

- JWT-based authentication
- Secure API calls using `Authorization: Bearer`
- Protected frontend routes
- Session/token handling
- Frontend input validation
- XSS-safe rendering
- Logout behavior
- Basic CSRF awareness

### Scenario

You are building the frontend for an internal notes application. Only authenticated users should be able to view or create notes. Notes may contain user input, so the frontend must render them safely.

### Part 1: Create the Pages

Create these frontend views:

```
/login  
/dashboard
```

The login page should have:

```
Username  
Password  
Login button  
Error message area
```

The dashboard should have:

```
Welcome message
List of notes
Add note form
Logout button
```

## Part 2: Mock the Backend

You can use fake API functions instead of a real backend.

Create these functions:

```
login(username, password)
getNotes(token)
addNote(token, noteText)
```

Example behavior:

```
const fakeToken = "mock.jwt.token";

function login(username, password) {
  if (username === "student" && password === "Password123") {
    return Promise.resolve({ token: fakeToken });
  }

  return Promise.reject(new Error("Invalid username or password"));
}
```

## Part 3: Store the Token

After login:

```
sessionStorage.setItem("accessToken", token);
```

Then redirect the user to the dashboard.

For this lab, use `sessionStorage` instead of `localStorage` so the token disappears when the browser session ends.

## Part 4: Protect the Dashboard

When the dashboard loads:

```
const token = sessionStorage.getItem("accessToken");

if (!token) {
  window.location.href = "/login";
}
```

Expected behavior:

```
No token -> redirect to login  
Valid token -> show dashboard
```

## Part 5: Make Secure API Calls

When fetching notes, pass the token:

```
getNotes(token);
```

If using real `fetch`, it would look like:

```
fetch("/api/notes", {  
  headers: {  
    Authorization: `Bearer ${token}`  
  }  
});
```

Practice handling:

```
200 OK -> show notes  
401 Unauthorized -> redirect to login  
500 Server Error -> show safe error message
```

## Part 6: Validate Note Input

Before allowing a note to be submitted:

Rules:

```
Note cannot be empty  
Note must be under 250 characters  
Note should not be only spaces
```

Example:

```
if (!noteText.trim()) {  
  showError("Note cannot be empty.");  
}
```

## Part 7: Render Notes Safely

Do not render notes with `innerHTML`.

Avoid:

```
noteElement.innerHTML = note.text;
```

Use:

```
noteElement.textContent = note.text;
```

Test with this note:

```
<img src=x onerror="alert('XSS')">
```

Expected result:

```
The text appears as text.  
No alert runs.
```

## Part 8: Add Logout

When the user clicks logout:

```
sessionStorage.removeItem("accessToken");  
window.location.href = "/login";
```

After logout, the dashboard should no longer be accessible.

## Part 9: Security Questions

Answer these after finishing:

1. Why should frontend validation also be repeated on the backend?
2. Why is `innerHTML` dangerous with user input?
3. What does the `Authorization: Bearer` header do?
4. Why should frontend code not contain private API secrets?
5. How does `HttpOnly` help protect cookies?
6. Why can cookie-based sessions need CSRF protection?

## Success Criteria

Your lab is complete when:

```
User can log in with valid credentials.  
Invalid login shows an error.  
Dashboard is blocked without a token.  
Notes are fetched using a token.  
Empty notes are rejected.  
XSS test input does not execute.  
Logout clears the session.  
Protected page cannot be viewed after logout.
```

## Optional Challenge

Add fake token expiration.

Example:

```
const token = {  
  value: "mock.jwt.token",  
  expiresAt: Date.now() + 5 * 60 * 1000  
};
```

Then make the dashboard redirect to login when the token expires.

---

## 10. Quick Check

Questions:

1. Should the frontend store private API secrets?
2. Is frontend validation enough to protect the backend?
3. Why is `innerHTML = userInput` dangerous?
4. Why do cookies need CSRF protection?
5. Who has final authority: frontend or backend?

Answers:

1. No.
2. No.
3. It may execute attacker-provided scripts.
4. Browsers send cookies automatically.
5. Backend.